

DOSSIER

Titre du dossier : Projet Optimisation et Applications

A L'attention de **Leandro MONTERO** :

Felix ASTOURY

Victor BOUSSEAU

Mohamed-Yacine ZNEDI

Elliot LANNUZEL

MASTER 2

Faculté des Sciences

Institut des mathématiques

Année universitaire : 2025 - 2026

Table des matières

Introduction	4
PLNE du TSP.....	5
Question 1 :	6
Question 2 : Algorithme Exact (Branch & Bound).....	7
2.1 Principe général	7
2.2 Pseudo-code (pour Carbon).....	8
2.3 Complexité	9
2.4 Instances de mauvaises qualités.....	9
Question 3.....	10
3.1 Principe général	10
3.2 Pseudo code.....	10
3.3 Complexité	11
3.4 Borne de performance (Ratio d'approximation)	11
Question 4.....	13
4.1 Principe général	13
4.2 Pseudo code.....	13
4.3 Complexité	14
4.4 Borne de performance.....	14
Question 5.....	15
5.1 Principe général	15
5.2 Pseudo code.....	16
5.3 Complexité	16
5.4 Choix et justification des paramètres.....	17
5.5 Borne de performance.....	17
Question 6 : Expérimentations par algorithme.....	18
6.1 Protocole expérimental	18
6.2 Résultats de l'algorithme Exact (Branch & Bound).....	18
6.3 Résultats de l'heuristique Constructive (Nearest Neighbor).....	18
6.4 Résultats de la Recherche Locale (2-Opt)	18

6.5 Résultats de la Métaheuristique GRASP	19
Question 7 : Comparaison finale et conclusions.....	20
7.1 Analyse comparative des coûts	20
7.2 Analyse comparative des gaps	21
7.3 Analyse des temps d'exécution (tous algorithmes).....	22
7.4 Analyse des temps d'exécution (heuristiques)	23
7.5 Synthèse et recommandations.....	23
7.6 Conclusions	24

Introduction

Le Problème du Voyageur de Commerce (Traveling Salesman Problem, TSP) est un problème classique d'optimisation combinatoire, étant donnée une liste de villes et les distances entre chaque paire, il s'agit de déterminer le plus court circuit passant exactement une fois par chaque ville avant de revenir au point de départ. Ce problème est connu pour être NP-difficile, ce qui implique qu'une résolution exacte devient rapidement coûteuse lorsque le nombre de villes augmente. Malgré cela, le TSP reste un problème central car il modélise de nombreux cas concrets (tournées de livraison, planification d'itinéraires, inspection de sites, routage en production, etc.) et sert de benchmark pour comparer des méthodes exactes et heuristiques.

L'objectif de ce projet est de développer, implémenter et analyser plusieurs approches de résolution du TSP, en suivant un axe progressif allant d'une méthode exacte vers des heuristiques et métaheuristiques plus adaptées aux grandes instances. Nous considérons un graphe complet pondéré représentant les distances entre villes, et nous cherchons un cycle hamiltonien de poids minimal. Quatre familles de méthodes sont étudiées : (I) un algorithme exact de type Branch & Bound, (II) une heuristique constructive gloutonne (Nearest Neighbor), (III) une heuristique de recherche locale (2-opt) pour améliorer une tournée initiale, et (IV) une métaheuristique GRASP combinant construction randomisée et amélioration locale.

Les algorithmes ont été implémentés en Python dans une structure modulaire. Un module "modèle" gère la lecture des instances (matrice .in et format TSPLIB .tsp), le calcul de coûts et l'export des solutions, tandis que chaque méthode dispose de son propre module. Les solutions produites sont comparées expérimentalement selon deux critères principaux, la qualité (coût de la tournée) et le temps d'exécution en faisant varier la taille des instances ainsi que les paramètres des heuristiques lorsque nécessaire (par exemple la taille de la liste restreinte de candidats et le nombre d'itérations pour GRASP). Enfin, le rapport discute les limites de chaque méthode (cas où l'optimalité n'est pas garantie) et met en évidence les compromis précision/temps de calcul observés.

PLNE du TSP

Le TSP peut être formulé comme un Programme Linéaire en Nombres Entiers (PLNE).

$$d_{ij} = \text{distance entre la ville } i \text{ et la ville } j$$

Variables de décision :

$$x_{ij} = \begin{cases} 1 & \text{si l'arc}(i, j) \text{ est emprunté} \\ 0 & \text{sinon} \end{cases}$$

$$u_i = \text{position de la ville } i \text{ dans la tournée } (i \geq 1)$$

Fonction objective :

$$\min \sum_{i=0}^{n-1} \sum_{j=0, j \neq i}^{n-1} d_{ij} \cdot x_{ij}$$

Contraintes :

Chaque ville est quittée exactement une fois :

$$\forall j \in \{0, \dots, n-1\}, \sum_{i \neq j} x_{ij} = 1$$

Chaque ville est atteinte exactement une fois :

$$\forall j \in \{0, \dots, n-1\}, \sum_{i \neq j} x_{ij} = 1$$

élimination des sous-tours :

$$\forall i, j \in \{1, \dots, n-1\}, i \neq j: u_i - u_j + (n-1) \cdot x_{ij} \leq n-2$$

Bornes sur les variables de position :

$$\forall i \in \{1, \dots, n-1\}: 2 \leq u_i \leq n$$

Intégrité des variables :

$$x_{ij} \in \{0,1\}$$

Nous avons donc finalement : $Z = \min \sum_{i=0}^{n-1} \sum_{j=0, j \neq i}^{n-1} d_{ij} \cdot x_{ij}$

$$S.C \left\{ \begin{array}{l} \forall i \in \{0, \dots, n-1\}, \sum_{i \neq j} x_{ij} = 1 \\ \forall j \in \{0, \dots, n-1\}, \sum_{i \neq j} x_{ij} = 1 \\ \forall i, j \in \{1, \dots, n-1\}, i \neq j: u_i - u_j + (n-1) \cdot x_{ij} \leq n-2 \\ \forall i \in \{1, \dots, n-1\}: 2 \leq u_i \leq n \\ x_{i,j} \in \{0,1\} \end{array} \right.$$

Question 1 :

Décrire (d'autres) situations réelles qui peuvent être modélisées comme un TSP.

1. Transformation de chaînes de caractères (Distance d'édition)

Étant donné un ensemble de mots, on cherche à passer par chaque mot en effectuant le moins d'opérations possible. Chaque mot représente une "ville" et la "distance" entre deux mots correspond à leur distance d'édition, c'est-à-dire le nombre minimal d'opérations élémentaires pour transformer un mot en un autre : substitution, insertion ou suppression d'une lettre. Par exemple, la distance entre "chat" et "chant" est de 1 (insertion du 'n'). Ce problème apparaît notamment en correction orthographique et en traitement automatique du langage.

2. Planification d'observations astronomiques

Un télescope doit observer un certain nombre d'objets célestes (étoiles, planètes, galaxies) durant une session d'observation. Chaque repositionnement de la lunette astronomique est coûteux en temps et en énergie. Les objets à observer constituent les "villes" et la "distance" entre deux cibles représente le temps ou l'énergie nécessaire pour réorienter le télescope (fonction de l'angle de rotation). Minimiser le trajet total permet de maximiser le temps effectif d'observation et de réduire l'usure des mécanismes de pointage.

3. Optimisation des déplacements dans un jeu vidéo (Among Us)

Dans le jeu vidéo Among Us, chaque joueur "coéquipier" doit accomplir une liste de tâches réparties sur différentes zones de la carte, tout en évitant les imposteurs. Chaque emplacement de tâche représente une "ville" et les "distances" correspondent aux temps de déplacement entre les différentes salles du vaisseau. Compléter ses tâches le plus rapidement possible (en minimisant le trajet total) augmente les chances de survie du joueur et les chances de victoire de l'équipe. Ce cas illustre comment le TSP peut s'appliquer à des contextes ludiques où l'efficacité des déplacements est cruciale.

Question 2 : Algorithme Exact (Branch & Bound)

2.1 Principe général

L'algorithme Branch & Bound (séparation et évaluation) est une méthode exacte permettant de résoudre le TSP de manière optimale. Le principe repose sur l'exploration intelligente de l'espace des solutions en construisant un arbre de recherche, tout en éliminant (élaguant) les branches qui ne peuvent pas contenir la solution optimale.

L'idée centrale est la suivante : à chaque nœud de l'arbre, on calcule une borne inférieure du coût de toute solution réalisable dans cette branche. Si cette borne dépasse le coût de la meilleure solution entière déjà trouvée, on peut ignorer toute la branche sans perdre l'optimalité.

L'algorithme utilise une relaxation linéaire du problème, on autorise les variables binaires à prendre des valeurs continues entre 0 et 1. La solution de cette relaxation fournit une borne inférieure. Si la solution est entière et forme un tour valide, on a trouvé une solution réalisable. Sinon, on branche en fixant une variable fractionnaire à 0 ou 1, créant deux sous-problèmes.

Ainsi, nous nous retrouvons avec le model suivant :

$$Z = \min \sum_{i=0}^{n-1} \sum_{j=0, j \neq i}^{n-1} d_{ij} \cdot x_{ij}$$
$$S.C \left\{ \begin{array}{l} \forall i \in \{0, \dots, n-1\}, \sum_{i \neq j} x_{ij} = 1 \\ \forall j \in \{0, \dots, n-1\}, \sum_{i \neq j} x_{ij} = 1 \\ \forall i, j \in \{1, \dots, n-1\}, i \neq j: u_i - u_j + (n-1) \cdot x_{ij} \leq n-2 \\ \forall i \in \{1, \dots, n-1\}: 2 \leq u_i \leq n \\ x_{i,j} \in [0,1] \end{array} \right.$$

2.2 Pseudo-code (pour Carbon)

```
ALGORITHME BranchAndBound(n, D)

ENTRÉES : n (nombre de villes), D (matrice des distances)
SORTIE  : meilleur_chemin, meilleur_coût

meilleur_coût ← +∞
meilleur_chemin ← NULL
file_priorité ← file vide (triée par borne croissante)

// Créer le problème de base (aucune variable fixée)
problème_initial ← CréerPLNE(n, D)
Ajouter (problème_initial, borne=0) à file_priorité

TANT QUE file_priorité ≠ ∅ FAIRE
|
|   // Séparation sur le noeud le plus prométeur,
|   // ie le plus proche de 0.5
|   (problème, _) ← ExtraireMin(file_priorité)
|
|   // Résolution la relaxation linéaire
|   solution ← RésoudreRelaxation(problème)
|
|   // Élagage
|   SI solution.coût ≥ meilleur_coût ALORS
|   |   Continuer
|   FIN SI
|
|   // Vérifier si solution entière et valide
|   SI EstEntière(solution) ET EstTourValide(solution) ALORS
|   |   meilleur_coût ← solution.coût
|   |   meilleur_chemin ← ExtraireChemin(solution)
|   SINON
|   |   var ← ChoisirVariableFractionnaire(solution)
|   |
|   |   // Créer deux sous-problèmes
|   |   problème_0 ← Fixer(problème, var = 0)
|   |   problème_1 ← Fixer(problème, var = 1)
|   |
|   |   Ajouter (problème_0, solution.coût) à file_priorité
|   |   Ajouter (problème_1, solution.coût) à file_priorité
|   FIN SI
|
FIN TANT QUE

RETOURNER meilleur_chemin, meilleur_coût
```


2.3 Complexité

En observant le bloc SINON à la fin du pseudo-code, on remarque que lorsque la solution n'est pas entière, l'algorithme exécute une dichotomie. Cela signifie qu'à chaque étape non résolue, un nœud de l'arbre de recherche génère deux enfants.

		<i>problème_0</i> ← <i>Fixer</i> (<i>problème</i> , <i>var</i> = 0)
		<i>problème_1</i> ← <i>Fixer</i> (<i>problème</i> , <i>var</i> = 1)

L'instruction

		<i>SI solution.coût</i> ≥ <i>meilleur_coût</i> ALORS
		<i>Continuer</i>

permet théoriquement de

couper des branches entières de l'arbre.

Cependant, dans le pire scénario possible, cette condition n'est jamais vérifiée avant de connaître tout l'arbre. L'algorithme est forcé d'explorer l'intégralité de l'arbre jusqu'aux feuilles pour valider chaque tour.

Si l'élagage échoue, l'algorithme doit générer et vérifier tous les tours hamiltoniens possibles pour trouver l'optimal. Pour un graphe complet de n villes :

- On peut choisir la première ville de n façons (mais comme c'est un cycle, le point de départ importe peu, on le fixe).
- Il reste $n-1$ choix pour la deuxième ville, $n-2$ pour la troisième, et ainsi de suite. Cela donne $(n-1)!$ permutations.
- Puisqu'un tour peut être parcouru dans les deux sens, on divise ce nombre par 2.

Le nombre total de feuilles à explorer est donc de $(n-1)!/2$

Puisque le nombre d'opérations est directement proportionnel au nombre de tours à explorer dans cet arbre, la complexité asymptotique finale dans le pire des cas s'exprime par $O(n!)$

Ce résultat confirme que, sans un élagage efficace, le temps de calcul explose de manière factorielle, rendant l'approche impraticable au-delà d'une vingtaine de villes.

2.4 Garantie de performance

Comme précisé dans « 2.1 Principe général » l'algorithme Branch & Bound est une méthode de résolution exacte. Par définition, son exploration systématique de l'espace des solutions, couplée au principe d'évaluation par bornes, garantit que la solution retournée est systématiquement l'optimum global du problème. Il n'existe donc aucune instance pour laquelle cette méthode fournirait une solution de "mauvaise qualité" ou s'écarterait de l'optimal (gap de 0%). La seule limite de cet algorithme réside dans sa complexité factorielle $O(n!)$.

Question 3

Développer et implémenter une heuristique constructive pour résoudre le TSP.

3.1 Principe général

L'heuristique du plus proche voisin (Nearest Neighbor) est une méthode constructive gloutonne pour le TSP. Le principe est simple, partant d'une ville initiale, on se déplace toujours vers la ville non visitée la plus proche, jusqu'à avoir parcouru toutes les villes, puis on retourne au point de départ.

Cette approche est dite "gloutonne" car à chaque étape, elle effectue le choix localement optimal (la ville la plus proche) sans considérer les conséquences futures de ce choix. L'algorithme construit ainsi une solution complète de manière incrémentale, en ajoutant une ville à chaque itération.

3.2 Pseudo code

```
ALGORITHME NearestNeighbor(n, D)


---


ENTRÉES : n, D (matrice)
SORTIE  : chemin

non_visités ← {1, ..., n-1}
courant ← 0
chemin ← [courant]

TANT QUE non_visités ≠ ∅ FAIRE
|   dist_min ← +∞
|   POUR CHAQUE v ∈ non_visités FAIRE
|   |   SI D[courant][v] < dist_min ALORS
|   |   |   dist_min ← D[courant][v]
|   |   |   plus_proche ← v
|   |   FIN SI
|   FIN POUR
|   courant ← plus_proche
|   Ajouter courant à chemin
Ajouter 0 à chemin
FIN TANT QUE

RETOURNER chemin
```

3.3 Complexité

Analysons la complexité à partir du pseudo-code :

Initialisation : $O(n)$ pour créer l'ensemble des villes non visitées.

La boucle principale, TANT QUE s'exécute exactement $(n-1)$ fois, car on ajoute une ville au chemin à chaque itération jusqu'à ce que toutes les villes soient visitées.

Dans la boucle interne, à l'itération k , l'ensemble non visités contient $(n-k)$ éléments. La recherche du plus proche voisin parcourt tous ces éléments.

Calcul total : Le nombre total d'opérations de comparaison est :

$$\sum_{k=1}^{n-1} (n-k) = (n-1) + (n-2) + \dots + 1 = \frac{n(n-1)}{2} = \frac{n^2 - n}{2}$$

Complexité finale : $O(n^2)$

Cette complexité quadratique rend l'algorithme très efficace même pour de grandes instances. Pour $n = 1000$ villes, l'algorithme effectue environ 500 000 comparaisons, ce qui s'exécute en quelques millisecondes sur un ordinateur moderne.

3.4 Borne de performance (Ratio d'approximation)

Bien que l'heuristique Nearest Neighbor soit rapide et intuitive, elle ne garantit pas de trouver la solution optimale. Plus précisément, il a été démontré théoriquement que la solution produite peut être jusqu'à $O(\log n)$ fois plus mauvaise que l'optimal.

Théorème (Rosenkrantz, Stearns, Lewis, 1977) : Pour le TSP métrique (où l'inégalité triangulaire est respectée), l'heuristique du plus proche voisin produit une tournée de coût au plus :

$$C_{NN} \leq \frac{1}{2} (\lceil \log_2 n \rceil + 1) \times C_{OPT}$$

où C_{NN} est le coût de la solution Nearest Neighbor et C_{OPT} est le coût optimal.

Intuition de cette borne :

Le problème fondamental de Nearest Neighbor est que l'algorithme fait toujours le meilleur choix local sans anticiper les conséquences. En fin de parcours, il peut rester des villes très éloignées du noeud courant, forçant de longs déplacements.

Considérons une construction piègeuse :

- On peut construire des instances où les premiers choix gloutons semblent excellents
- Mais ces choix "piègent" l'algorithme en laissant des villes isolées pour la fin

- Le retour au point de départ peut également être très coûteux

Le facteur logarithmique provient du fait que l'on peut construire des instances où chaque "mauvais choix forcé" double approximativement le coût par rapport à l'optimal, et cela peut se produire $O(\log n)$ fois.

En pratique, sur des instances aléatoires ou réelles, Nearest Neighbor performe généralement bien mieux que cette borne théorique pessimiste. Nos expérimentations montrent des gaps moyens de 15-25% par rapport à la meilleure solution connue, ce qui est acceptable pour une heuristique aussi simple et rapide.

Question 4

Développer et implémenter une heuristique de recherche locale pour résoudre le TSP.

4.1 Principe général

L'algorithme 2-Opt est une heuristique d'amélioration locale pour le TSP. Contrairement à l'heuristique constructive qui construit une solution de zéro, 2-Opt part d'une solution existante (par exemple celle produite par Nearest Neighbor) et tente de l'améliorer itérativement.

Le principe est simple, on examine toutes les paires d'arêtes non adjacentes de la tournée et on teste si leur "échange" produit une meilleure solution. Concrètement, si on a une tournée passant par les arêtes (u,v) et (x,y) , on teste si remplacer ces arêtes par (u,x) et (v,y) réduit le coût total. Si oui, cela revient à inverser le segment de la tournée situé entre v et x .

L'algorithme répète ce processus jusqu'à ce qu'aucun échange 2-Opt n'améliore plus la solution. On dit alors que la solution est "2-optimale" (aucun échange de 2 arêtes ne peut l'améliorer).

4.2 Pseudo code

```
ALGORITHME 2-Opt(chemin, D)

ENTRÉES : chemin (tournée initiale), D (matrice des distances)
SORTIE : chemin amélioré

n ← taille(chemin)
amélioré ← VRAI
meilleur_chemin ← copie(chemin)

TANT QUE amélioré FAIRE
    amélioré ← FAUX
    POUR i DE 0 À n-2 FAIRE
        POUR j DE i+2 À n-1 FAIRE
            u ← meilleur_chemin[i]
            v ← meilleur_chemin[i+1]
            x ← meilleur_chemin[j]
            y ← meilleur_chemin[(j+1) mod n]

            coût_actuel ← D[u][v] + D[x][y]
            coût_nouveau ← D[u][x] + D[v][y]

            SI coût_nouveau < coût_actuel ALORS
                Inverser segment [i+1, j] dans meilleur_chemin
                amélioré ← VRAI
            FIN SI
        FIN POUR
    FIN POUR
FIN TANT QUE

RETOURNER meilleur_chemin
```

4.3 Complexité

Analyse d'une itération de la boucle TANT QUE :

Les deux boucles FOR imbriquées examinent toutes les paires (i, j) avec $0 \leq i < n-1$ et $i+2 \leq j < n$.

Le nombre de paires examinées est :

$$\sum_{i=0}^{n-2} \sum_{j=i+2}^{n-1} 1 = \sum_{i=0}^{n-2} (n-2-i) = \frac{(n-1)(n-2)}{2}$$

Chaque comparaison et inversion potentielle se fait en $O(n)$ dans le pire cas (pour l'inversion du segment).

La complexité d'une itération est $O(n^2)$ comparaisons $\times O(n)$ pour l'inversion = **$O(n^3)$** dans le pire cas.

Le nombre d'itérations de la boucle TANT QUE n'est pas borné théoriquement, mais en pratique il est généralement faible (souvent $O(n)$ itérations).

Complexité totale observée : $O(n^2)$ à $O(n^3)$ en pratique.

4.4 Borne de performance

L'algorithme 2-Opt garantit de trouver un optimum local, c'est-à-dire une solution qui ne peut pas être améliorée par l'échange de deux arêtes. Cependant, cet optimum local n'est pas nécessairement l'optimum global (la meilleure solution possible).

Il peut exister plusieurs optima locaux de qualités différentes, et l'algorithme convergera vers l'un d'entre eux selon la solution initiale fournie. C'est pourquoi 2-Opt est souvent combiné avec d'autres techniques (comme GRASP) qui génèrent plusieurs solutions initiales différentes pour explorer davantage l'espace des solutions.

2-Opt échoue à sortir d'un optimum local sur des instances où l'échange de deux arêtes ne suffit pas à sortir du dit optimum.

Un exemple est l'instance où une tournée est composée de deux sous-ensembles de villes reliés entre eux. Le 2-Opt peut rester bloqué dans un optimum local où les connexions entre ces clusters sont sous-optimales, car pour améliorer la solution, il serait nécessaire de modifier quatre arêtes simultanément. Comme le 2-Opt est limité à la modification de deux arêtes, toute tentative d'amélioration locale augmente le coût total, emprisonnant l'algorithme dans une solution de moins bonne qualité que l'optimum global.

Question 5

- Développer et implémenter un algorithme utilisant la méta-heuristique GRASP [1] pour le TSP.
- À chaque itération du GRASP, utiliser comme première phase une modification de l'heuristique donnée en 3, et comme seconde phase l'heuristique donnée en 4.
- FIXER tous les paramètres impliqués (taille de la liste restreinte de candidats (RLC), critères d'arrêt, etc.) via des expériences.
- JUSTIFIER vos choix et le choix des instances de test utilisées pour fixer vos paramètres.

5.1 Principe général

GRASP (Greedy Randomized Adaptive Search Procedure) est une métaheuristique itérative qui combine deux phases complémentaires à chaque itération :

1. Phase de construction randomisée qui génère une solution réalisable en introduisant de l'aléatoire dans une heuristique gloutonne (ici, Nearest Neighbor)
2. Phase d'amélioration locale qui améliore la solution construite à l'aide d'une recherche locale (ici, 2-Opt)

L'idée fondamentale de GRASP est de répéter ce processus plusieurs fois avec des choix aléatoires différents, permettant ainsi d'explorer diverses régions de l'espace des solutions. La meilleure solution trouvée parmi toutes les itérations est conservée comme résultat final.

La randomisation de la phase constructive est réalisée via une Liste Restreinte de Candidats (RCL). Au lieu de toujours choisir le meilleur candidat (approche purement gloutonne), on sélectionne aléatoirement parmi les α meilleurs candidats. Le paramètre α contrôle le compromis entre qualité ($\alpha=1$, glouton pur) et diversification (α grand, plus aléatoire).

5.2 Pseudo code

```
FONCTION ConstruireRCL(n, D, α, départ)
// Nearest Neighbor avec Liste Restreinte de Candidats

non_visités ← {0, 1, ..., n-1}
courant ← départ
chemin ← [courant]
Retirer courant de non_visités

TANT QUE non_visités ≠ ∅ FAIRE
    // Construire liste des candidats triés par distance
    candidats ← [(D[courant][v], v) pour v ∈ non_visités]
    Trier candidats par distance croissante

    // Construire la RCL (α meilleurs)
    taille_rcl ← min(α, |candidats|)
    RCL ← candidats[0 : taille_rcl]

    // Choisir aléatoirement dans la RCL
    (_, choisi) ← ALÉATOIRE(RCL)

    courant ← choisi
    Ajouter courant à chemin
    Retirer courant de non_visités
FIN TANT QUE

RETOURNER chemin
```

5.3 Complexité

Complexité d'une itération GRASP :

- Phase de construction (ConstruireRCL) : $O(n^2 \log n)$ due au tri à chaque étape
- Phase d'amélioration (2-Opt) : $O(n^2)$ à $O(n^3)$ selon le nombre de passes

Complexité totale :

$$C_{GRASP} = max_iter \times (O(n^2 \log n) + O(n^3)) = O(max_iter \times n^3)$$

Étant donné que max_iter est une constante, (par exemple 30), la complexité devient :

$$C_{GRASP} = O(30 \times n^3) = O(n^3)$$

5.4 Choix et justification des paramètres

Les paramètres de GRASP ont été fixés selon les considérations suivantes :

Paramètre α (taille de la RCL) : $\alpha = 2$

Le paramètre α contrôle le degré de randomisation de la phase constructive :

1. $\alpha = 1$: comportement purement glouton (Nearest Neighbor classique), aucune diversification
2. α trop grand : choix trop aléatoires, solutions initiales de mauvaise qualité

Avec $\alpha = 2$, à chaque étape de construction, on choisit aléatoirement parmi les 2 villes les plus proches. Ce choix offre un bon compromis :

- Suffisamment de diversification pour explorer différentes solutions
- Solutions construites restant de qualité raisonnable avant l'amélioration 2-Opt

Nombre d'itérations : $\text{max_iter} = 30$

Le nombre d'itérations détermine l'effort de recherche global :

- Trop peu d'itérations : exploration insuffisante, risque de manquer de bonnes solutions
- Trop d'itérations : temps de calcul excessif pour des gains marginaux

Avec 30 itérations, on génère 30 solutions différentes grâce à la randomisation. Ce nombre permet généralement d'atteindre une convergence satisfaisante tout en maintenant un temps d'exécution raisonnable (quelques secondes pour $n = 500$).

À chaque itération, le nœud de départ est choisi aléatoirement parmi les n villes. Cette stratégie augmente la diversification en permettant d'explorer des tournées structurellement différentes, complétant ainsi la randomisation de la RCL.

5.5 Borne de performance

GRASP est une métaheuristique qui ne garantit pas de trouver la solution optimale. Cependant, en combinant construction randomisée et amélioration locale sur plusieurs itérations, GRASP explore plus efficacement l'espace des solutions que 2-Opt seul.

Là où 2-Opt partant d'une unique solution Nearest Neighbor converge vers un seul optimum local, GRASP génère et améliore 30 solutions différentes, augmentant ainsi la probabilité de découvrir un meilleur optimum local.

Nos expérimentations (voir section 6) montrent que GRASP produit systématiquement les meilleures solutions parmi les heuristiques testées, avec des gaps généralement inférieurs à 5% par rapport à la meilleure solution connue.

Question 6 : Expérimentations par algorithme

6.1 Protocole expérimental

Les expérimentations ont été réalisées sur des instances aléatoires générées pour des tailles allant de $n = 5$ à $n = 500$ villes. Pour chaque taille, une instance est générée puis résolue par les quatre algorithmes implémentés. Les métriques mesurées sont le coût de la solution et le temps d'exécution.

Note sur le calcul du GAP : Pour les instances où l'algorithme exact a été exécuté ($n \leq 12$), le gap est calculé par rapport à la solution optimale. Pour les instances plus grandes, le gap est calculé par rapport à la meilleure solution trouvée parmi les heuristiques, celle-ci servant alors de référence.

6.2 Résultats de l'algorithme Exact (Branch & Bound)

L'algorithme exact n'a été exécuté que pour les instances de petite taille ($n \leq 12$) en raison de sa complexité exponentielle. Les résultats montrent que :

- Le temps d'exécution croît très rapidement avec n (visible sur le graphique des temps en échelle logarithmique)
- Pour $n = 12$, le temps atteint déjà plusieurs dizaines de secondes
- La solution retournée est garantie optimale (gap = 0%)

Au-delà de $n = 12$, l'algorithme devient impraticable dans un temps raisonnable, justifiant le recours aux heuristiques.

6.3 Résultats de l'heuristique Constructive (Nearest Neighbor)

L'heuristique Nearest Neighbor présente les caractéristiques suivantes :

- Temps d'exécution : Extrêmement rapide, quasi-instantané même pour $n = 500$ (de l'ordre de 10^{-2} secondes)
- Qualité des solutions : Les gaps observés varient entre 10% et 40% selon les instances, avec une forte variabilité
- Comportement : La courbe des coûts est systématiquement au-dessus des autres heuristiques

La rapidité de cet algorithme en fait une bonne solution pour obtenir une première solution réalisable, mais sa qualité reste insuffisante pour une utilisation finale.

6.4 Résultats de la Recherche Locale (2-Opt)

L'algorithme 2-Opt, partant de la solution Nearest Neighbor, montre une amélioration significative :

- Temps d'exécution est Modéré, croissant de manière polynomiale (quelques dixièmes de seconde pour $n = 500$)
- Qualité des solutions : Gaps généralement entre 2% et 15%, nettement meilleurs que Nearest Neighbor
- Comportement : La courbe des coûts se rapproche de celle de GRASP

2-Opt offre un bon compromis entre temps de calcul et qualité de solution pour des applications nécessitant une réponse rapide.

6.5 Résultats de la Métaheuristique GRASP

GRASP avec les paramètres calibrés ($\alpha = 2$, 30 itérations) obtient les meilleurs résultats :

- Temps d'exécution est le plus élevé parmi les heuristiques (environ 10 secondes pour $n = 500$), mais reste raisonnable
- Sur la qualité des solutions, gaps généralement inférieurs à 5%, souvent le meilleur résultat
- La courbe des coûts est souvent la plus basse, servant fréquemment de référence pour le calcul des gaps

Le surcoût en temps de calcul est largement compensé par la qualité supérieure des solutions obtenues.

Question 7 : Comparaison finale et conclusions

7.1 Analyse comparative des coûts

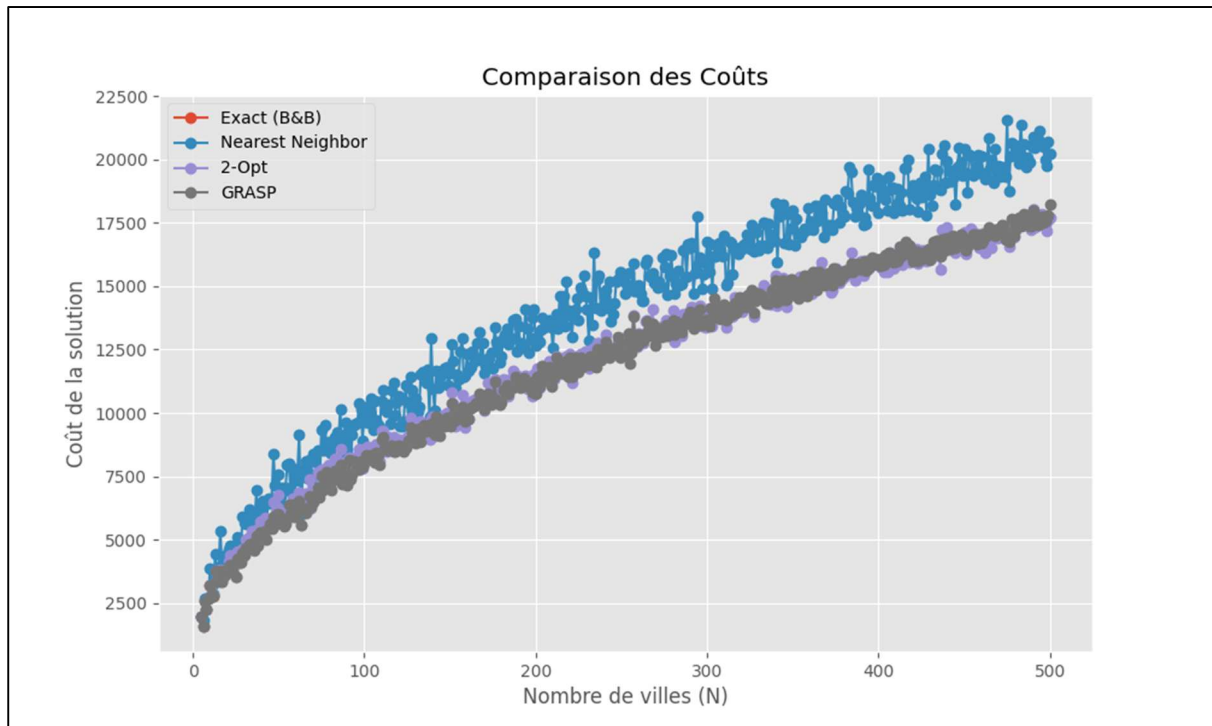


Figure 1 - Comparaison des coûts des solutions

Ce graphique montre l'évolution du coût des solutions en fonction du nombre de villes. On observe clairement deux groupes. Nearest Neighbor (bleu) produit systématiquement les solutions les plus coûteuses avec une forte dispersion. 2-Opt (violet) et GRASP (gris) produisent des solutions de qualité comparable et nettement meilleures, leurs courbes étant quasi superposées. L'écart entre Nearest Neighbor et les autres méthodes s'accroît avec la taille des instances.

7.2 Analyse comparative des gaps

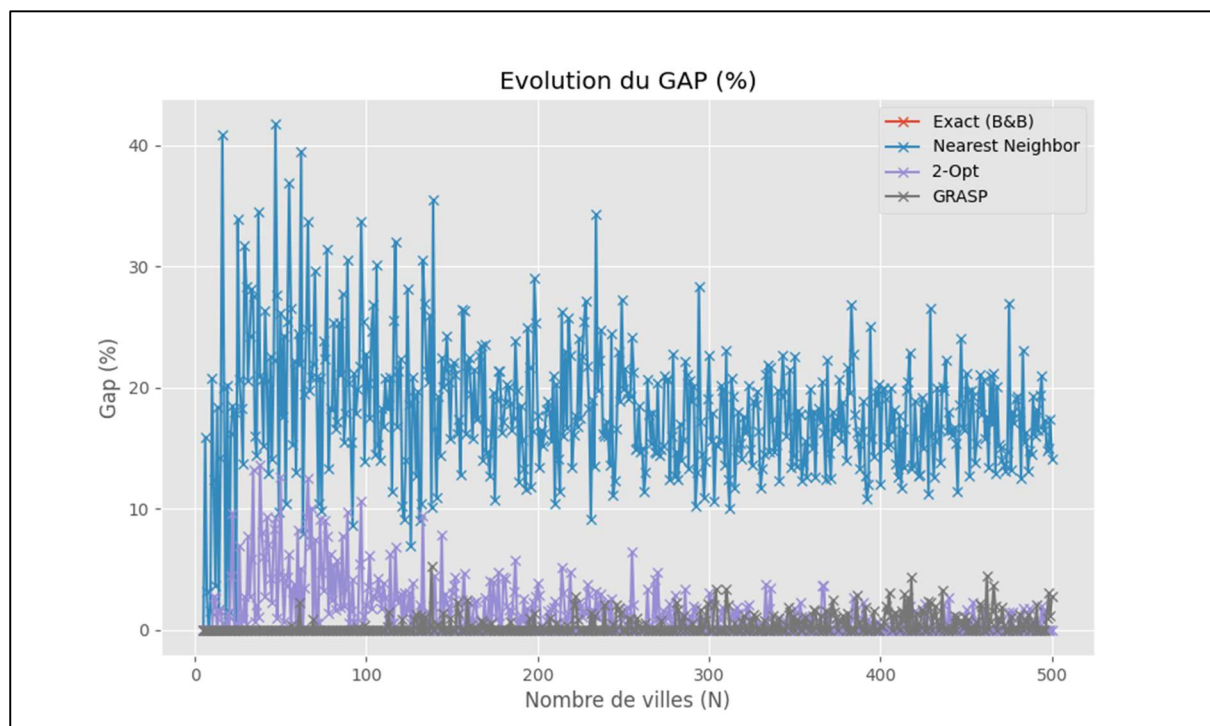


Figure 2 - Évolution du gap par rapport à la meilleure solution connue

Ce graphique illustre la qualité relative des solutions, c'est-à-dire la qualité des résultats par rapport au meilleur résultat fournis par une des heuristiques. Nearest Neighbor présente des gaps très variables (10-40%) et instables. 2-Opt maintient des gaps plus contenus (2-15%) avec moins de variance. GRASP affiche les gaps les plus faibles et les plus stables (0-5%), atteignant fréquemment le gap de 0% (meilleure solution trouvée). L'algorithme exact, lorsqu'il est exécuté ($n \leq 12$), garantit un gap nul correspondant à l'optimal absolu.

7.3 Analyse des temps d'exécution (tous algorithmes)

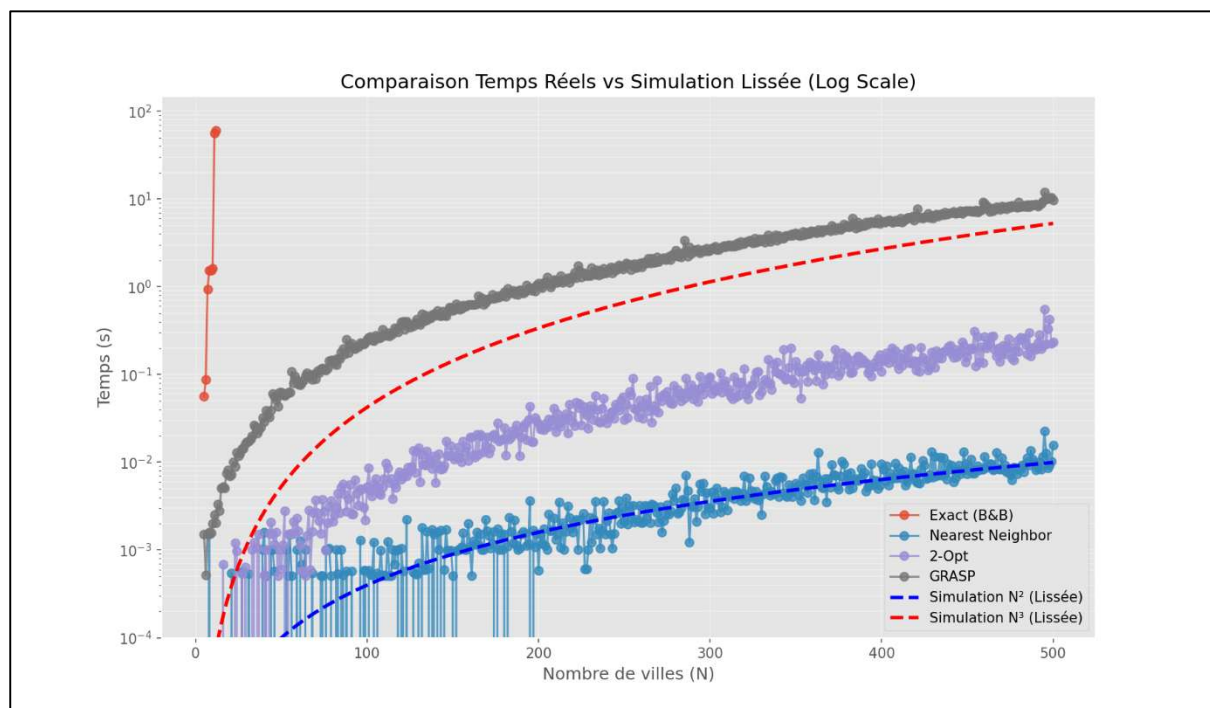


Figure 3 - Temps d'exécution en échelle logarithmique

L'échelle logarithmique permet de visualiser les différences d'ordre de grandeur entre les algorithmes. L'algorithme exact (rouge) montre une explosion du temps de calcul, passant de 10^{-1} à 10^2 secondes pour n allant de 5 à 12 seulement. Les heuristiques restent dans des temps raisonnables : GRASP (gris) autour de 10 secondes pour $n = 500$, 2-Opt (violet) autour de 0.5 seconde, et Nearest Neighbor (bleu) quasi-instantané.

Pour les courbes de simulation, nous avons lancé des boucles pour générer une complexité de n^2 et n^3 puis nous avons lissé ces courbe pour avoir un résultat plus visible.

7.4 Analyse des temps d'exécution (heuristiques)

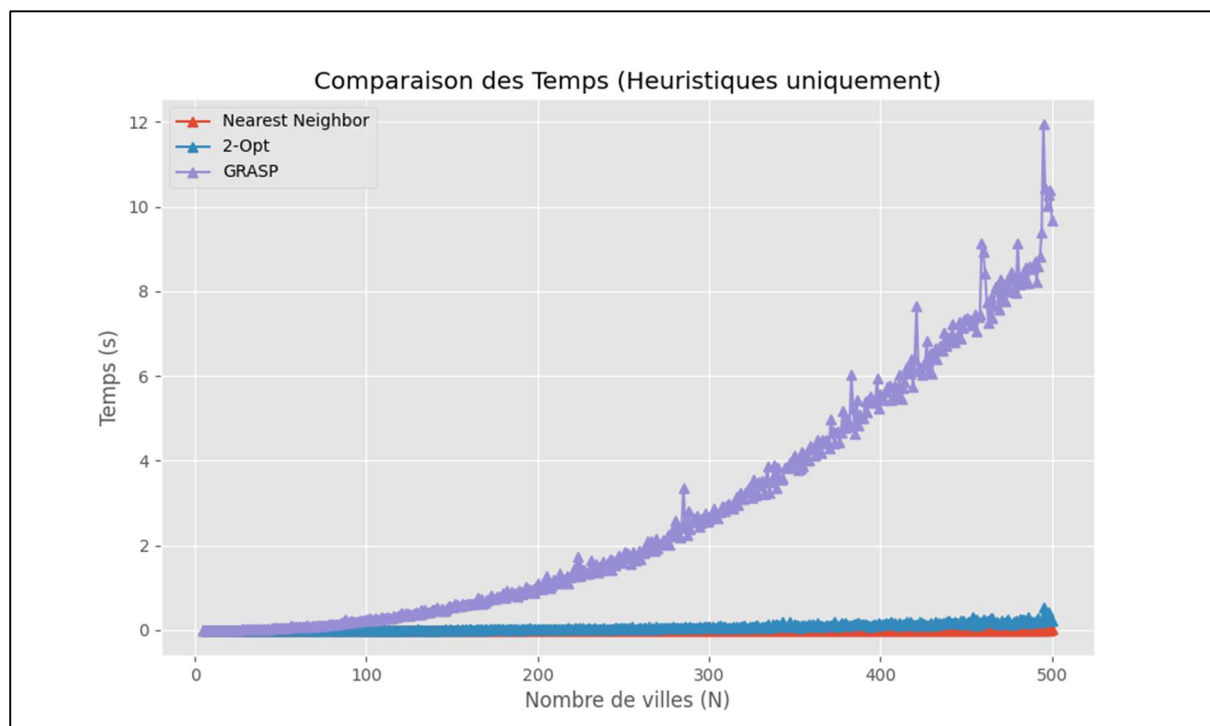


Figure 4 - Temps d'exécution des heuristiques

Sans l'algorithme exact, on distingue mieux les différences entre heuristiques. Nearest Neighbor reste négligeable en temps. 2-Opt croît modérément, atteignant environ 0.5 seconde pour $n = 500$. GRASP est le plus coûteux en temps (environ 10-12 secondes pour $n = 500$) en raison des 30 itérations effectuées, mais cette augmentation reste acceptable au regard de la qualité obtenue.

7.5 Synthèse et recommandations

Algorithme	Complexité	Temps (n=500)	Gap typique
Exact (B&B)	Exponentiel	Impraticable	0%
Nearest Neighbor	$O(n^2)$	~0.01s	15-40%
2-Opt	$O(n^3)$	~0.5s	2-15%
GRASP	$O(n^3)$	~10s	0-5%

7.6 Conclusions

Les expérimentations menées permettent de tirer plusieurs conclusions.

L'algorithme exact est limité aux très petites instances. Au-delà de 12-15 villes, le temps de calcul devient colossal. Son utilisation se justifie uniquement lorsqu'une solution optimale prouvée est indispensable et que la taille du problème le permet.

GRASP offre le meilleur compromis qualité/temps pour les instances moyennes et grandes. Avec des gaps généralement inférieurs à 5% et des temps de calcul de l'ordre de quelques secondes, GRASP est recommandé lorsque la qualité de la solution prime sur la rapidité.

L'algorithme 2-Opt constitue une alternative efficace lorsque le temps est contraint. Avec des temps dix fois inférieurs à GRASP pour une qualité légèrement moindre, 2-Opt convient aux applications temps-réel ou aux très grandes instances.

Nearest Neighbor seul est insuffisant. Bien que très rapide, la qualité des solutions (gaps de 15-40%) ne justifie son utilisation que comme solution initiale pour les autres méthodes.

La combinaison des approches est bénéfique. L'architecture de GRASP, combinant construction randomisée et amélioration locale, démontre l'intérêt de combiner plusieurs stratégies plutôt que de se reposer sur une seule heuristique.